Report: Permessi Linux

Introduzione

La gestione dei permessi in ambiente Linux rappresenta uno degli aspetti fondamentali della sicurezza informatica. Un corretto sistema di controllo degli accessi garantisce che solo gli utenti autorizzati possano leggere, modificare o eseguire determinati file e directory, proteggendo così l'integrità e la riservatezza dei dati. In questo esercizio pratico, viene simulato uno scenario reale di gestione dei log di sicurezza, dove è necessario configurare i permessi in modo da garantire la tracciabilità degli accessi al sistema, mantenendo al contempo la protezione contro modifiche non autorizzate.Lo scenario proposto prevede la creazione di un file di log degli accessi e di una directory dedicata ai log di sicurezza, entrambi configurati con permessi appropriati per bilanciare accessibilità e protezione

Obiettivi

Gli obiettivi di questo esercizio sono:
1. **Comprendere il sistema di permessi Linux:** Analizzare la struttura dei permessi e le operazioni consentite.
2. **Applicare configurazioni di sicurezza;**
3. **Utilizzare i comandi di gestione permessi;**
4. **Testare l'efficacia delle configurazioni:** Verificare praticamente l'effetto delle modifiche ai permessi attraverso operazioni di lettura, scrittura ed esecuzione

Svolgimento

Creazione del File e della Directory

Per questo esercizio sono stati creati un file di log e una directory dedicata ai log di sicurezza. Il file **access_log.txt** simula un registro degli accessi al sistema, mentre la directory s**ecurity_logs/** rappresenta una cartella che potrebbe contenere diversi file di log relativi alla sicurezza.

Comandi eseguiti:

touch access_log.txt ***mkdir security_logs***

```
kali@kali: ~
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ touch access_logs.txt

┌──(kali㉿kali)-[~]
└─$ mkdir security_logs
```

Popolazione file

In questa fase è stato anche popolato il file access_logs.txt con alcuni dati di esempio per rendere più realistico lo scenario.

```
┌──(kali㉿kali)-[~]
└─$  echo "Popolamento del file access_log.txt con dati di esempio..."
cat > access_logs.txt << 'EOF'
2026-02-10 08:15:23 - User marco logged in from 192.168.1.100
2026-02-10 08:47:11 - User giulia logged in from 192.168.1.105
2026-02-10 09:12:45 - User admin logged in from 192.168.1.10
2026-02-10 10:03:31 - User marco logged out
2026-02-10 11:22:08 - Failed login attempt for user test from 203.0.113.42
EOF
echo "✓ File popolato con dati di esempio"
echo ""

Popolamento del file access_log.txt con dati di esempio...
✓ File popolato con dati di esempio
```

Verifica dei Permessi Attuali

Prima di procedere con le modifiche, è stata effettuata una verifica dei permessi predefiniti assegnati automaticamente dal sistema al momento della creazione. Questo passaggio è fondamentale per comprendere lo stato iniziale e pianificare le modifiche necessarie.
Comandi eseguiti:

ls -l access_logs.txt ***ls -ld security_logs***

```
┌──(kali㉿kali)-[~]
└─$ ls -l access_logs.txt

-rw-rw-r-- 1 kali kali 305 Feb 10 10:27 access_logs.txt

┌──(kali㉿kali)-[~]
└─$ ls -ld security_logs
drwxrwxr-x 2 kali kali 4096 Feb 10 10:18 security_logs
```

I permessi predefiniti mostrano che sia il proprietario che il gruppo hanno permessi di lettura e scrittura sul file, mentre per la directory hanno anche il permesso di esecuzione. Gli altri utenti hanno solo permessi di lettura.

Modifica dei Permessi

In base allo scenario di sicurezza proposto, sono stati configurati i permessi per garantire un equilibrio tra accessibilità e protezione. Le scelte effettuate riflettono best practice comuni nella gestione dei log di sistema.

Configurazione per access_logs.txt:

```
┌──(kali㉿kali)-[~]
└─$ chmod 640 access_logs.txt

┌──(kali㉿kali)-[~]
└─$ chmod 750 security_logs
```

chmod 640 access_logs.txt: Il proprietario (amministratore di sistema) può leggere e aggiornare il log. Il gruppo può solo consultarlo per audit e analisi. Gli altri utenti non hanno alcun accesso, proteggendo informazioni potenzialmente sensibili.
Spiegazione del valore 640:
- 6 (proprietario): rw- = 4(lettura) + 2(scrittura) + 0(no esecuzione)
- 4 (gruppo): r-- = 4(lettura) + 0(no scrittura) + 0(no esecuzione)
- 0 (altri): --- = nessun permesso

chmod 750 security_logs: Il proprietario ha controllo completo (lettura, scrittura, esecuzione) per gestire i file di log. Il gruppo può navigare nella directory e leggere i file esistenti (r-x), ma non può creare, modificare o eliminare file. Gli altri utenti sono completamente esclusi dall'accesso.
Spiegazione del valore 750:
- 7 (proprietario): rw- = 4(lettura) + 2(scrittura) + 1(esecuzione)
- 5(gruppo): r-- = 4(lettura) + 0(no scrittura) + 1(esecuzione)
- 0 (altri): --- = nessun permesso

Verifica nuovi permessi

```
┌──(kali㉿kali)-[~]
└─$ ls -l access_logs.txt

-rw-r----- 1 kali kali 305 Feb 10 10:27 access_logs.txt

┌──(kali㉿kali)-[~]
└─$ ls -ld security_logs
drwxr-x--- 2 kali kali 4096 Feb 10 10:18 security_logs
```

Test dei permessi

Per verificare l'efficacia della configurazione dei permessi, sono stati eseguiti diversi test pratici simulando operazioni tipiche che potrebbero essere effettuate dagli utenti del sistema.

Test 1: Scrittura nel file access_log.txt .
Come proprietario del file, è stato possibile aggiungere nuove righe al log:

```
┌──(kali㉿kali)-[~]
└─$ echo "Test 1: Scrittura nel file access_logs.txt"
echo 'Comando: echo "2026-02-10 14:23:15 - User admin logged in from 192.168.1.45" >> access_log.t
xt'
echo "2026-02-10 14:23:15 - User admin logged in from 192.168.1.45" >> access_logs.txt
echo "✓ Scrittura riuscita (proprietario ha permesso rw-)"
echo ""

Test 1: Scrittura nel file access_logs.txt
Comando: echo "2026-02-10 14:23:15 - User admin logged in from 192.168.1.45" >> access_log.txt
✓ Scrittura riuscita (proprietario ha permesso rw-)
```

Test 2: Creazione di un nuovo file nella directory security_logs/
Tentativo di creare un file di log aggiuntivo all'interno della directory:

```
┌──(kali㉿kali)-[~]
└─$ touch security_logs/firewall_log.txt

┌──(kali㉿kali)-[~]
└─$ ls -l security_logs/firewall_log.txt
-rw-rw-r-- 1 kali kali 0 Feb 10 10:45 security_logs/firewall_log.txt
```

File creato con successo perché il proprietario ha i permessi di scrittura.

Test 3: Verifica lettura da parte del gruppo.
Simulazione della lettura del file da parte di un utente appartenente al gruppo autorizzato:

```
┌──(kali㉿kali)-[~]
└─$ cat access_logs.txt
2026-02-10 08:15:23 - User marco logged in from 192.168.1.100
2026-02-10 08:47:11 - User giulia logged in from 192.168.1.105
2026-02-10 09:12:45 - User admin logged in from 192.168.1.10
2026-02-10 10:03:31 - User marco logged out
2026-02-10 11:22:08 - Failed login attempt for user test from 203.0.113.42
2026-02-10 14:23:15 - User admin logged in from 192.168.1.45
```

Analisi dei risultati e conclusioni

I test effettuati hanno confermato che la configurazione dei permessi funziona come previsto, garantendo un equilibrio ottimale tra sicurezza e usabilità. L'utilizzo della notazione ottale ha permesso di impostare i permessi in modo rapido ed efficiente.

Vantaggi della configurazione adottata

- **Protezione dell'integrità dei dati**: Solo il proprietario può modificare i file di log, riducendo il rischio di alterazioni accidentali o malevole dei dati di audit.

- **Riservatezza:** Gli utenti non autorizzati (others) non possono accedere ai log, proteggendo informazioni sensibili come indirizzi IP, orari di accesso e attività degli utenti.

- **Conformità alle best practice:** La configurazione segue il principio del minimo privilegio (Principle of Least Privilege), assegnando solo i permessi strettamente necessari a ciascun livello.

- **Prevenzione di escalation dei privilegi**: Limitando l'accesso in scrittura, si riduce la superficie di attacco e si prevengono tentativi di manipolazione dei log per nascondere attività sospette.